STEP 1: Install Required Software (Do this FIRST)

✅ **1. Install Java (JDK 17)**

1. Go to Google → search:
   👉 JDK 17 download
2. Download and install
3. After install, open **Command Prompt** and type:

java -version

✓ You should see version

✅ **2. Install Maven**

1. Search: Apache Maven download
2. Download ZIP → Extract
3. Set PATH (important)

👉 After install, check:

mvn -version

✅ **3. Install Docker Desktop**

1. Download Docker Desktop
2. Open it
3. Go to ⚙ Settings:
   - ✅ Enable Kubernetes
4. Wait until it shows **Running**

✅ **4. Install Git**

Check:

git --version

✅ **5. Install Jenkins**

1. Download Jenkins
2. Start Jenkins
3. Open browser:

http://localhost:8080

4. Unlock Jenkins (copy password from screen)

🔴 **STEP 2: Create Project Folder**

📁 **Do this manually**

👉 Go to Desktop
👉 Right click → New Folder

Name:

prescription-app

📁 **Create subfolders**

Inside it create:

```
prescription-app
 ├── src
 │   ├── main
 │   │   └── java
 │   └── test
 │       └── java
```

👉 Use right click → New Folder each time

🔴 **STEP 3: Create Java Files**

📄 **1. Create Main File**

👉 Go to:

src/main/java

👉 Right click → New → Text File
Rename to:

PrescriptionService.java

👉 Open it → paste:

```
public class PrescriptionService {


   public static boolean isValidMedicine(String medicine) {

      if (medicine == null || medicine.trim().isEmpty()) return false;
```

```
    String[] meds = {"Paracetamol", "Ibuprofen"};
    for (String m : meds) {
        if (m.equalsIgnoreCase(medicine)) return true;
    }
    return false;
}

public static boolean isValidDosage(int dosage) {
    return dosage > 0 && dosage <= 1000;
}

public static boolean validatePrescription(String med, int dose) {
    return isValidMedicine(med) && isValidDosage(dose);
}

// 🔥 MAIN METHOD (IMPORTANT)
public static void main(String[] args) {
    System.out.println("Prescription App Running...");

    boolean result = validatePrescription("Paracetamol", 500);
    System.out.println("Validation Result: " + result);

    // Keep container alive
    while (true) {
        try {
            Thread.sleep(10000);
            System.out.println("App is running...");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

```
  }
}
```

👉 Save file

📄 **2. Create Test File**

👉 Go to:

src/test/java

👉 Create file:

PrescriptionTest.java

👉 Paste:

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class PrescriptionTest {

  @Test
  void testMedicine() {
    assertTrue(PrescriptionService.isValidMedicine("Paracetamol"));
    assertFalse(PrescriptionService.isValidMedicine("Wrong"));
  }

  @Test
  void testDosage() {
    assertTrue(PrescriptionService.isValidDosage(500));
    assertFalse(PrescriptionService.isValidDosage(2000));
  }
}
```

👉 Save

🔴 **STEP 4: Create pom.xml**

👉 In main folder (prescription-app)

👉 Create file:

pom.xml

👉 Paste:

```
<project xmlns="http://maven.apache.org/POM/4.0.0">

  <modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.demo</groupId>
<artifactId>prescription-app</artifactId>
<version>1.0</version>

<properties>
<maven.compiler.source>17</maven.compiler.source>
<maven.compiler.target>17</maven.compiler.target>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<junit.version>5.10.1</junit.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>
</dependencies>
<build>
<plugins>
  <!-- Make JAR executable -->
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-jar-plugin</artifactId>
    <version>3.3.0</version>
    <configuration>
      <archive>
        <manifest>
          <mainClass>PrescriptionService</mainClass>
```

```
            </manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
</build>
</project>
```

👉 Save

🔴 **STEP 5: Run Project (VERY IMPORTANT)**

👉 Open Command Prompt

👉 Go to project:

```
cd Desktop/prescription-app
```

👉 Run:

```
mvn clean test
```

✓ If SUCCESS → good
❌ If error → tell me

🔴 **STEP 6: Create Dockerfile**

👉 In main folder

Create:

Dockerfile

👉 Paste:

```
FROM eclipse-temurin:17-jdk-alpine
WORKDIR /app
COPY target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

🔴 **STEP 7: Build JAR**

Run:

mvn package

✓ This creates:

target/prescription-app-1.0.jar

🔴 **STEP 8: Build Docker Image**

👉 Run:

docker build -t yourname/prescription-app .

✓ Created a **Docker image** of your app
✓ Stored it locally in Docker

🔍 **VERIFY YOUR IMAGE**

Run this:

docker images

👉 You should see:

yourname/prescription-app latest

🚨 **IMPORTANT NEXT STEP (DON'T SKIP)**

Right now your image name is:

yourname/prescription-app

❌ This is just a placeholder
👉 You must replace **yourname** with your **DockerHub username**

🔴 **STEP 1: Fix Image Name**

👉 **Example**

If your DockerHub username = nivetha123

Run:

docker tag yourname/prescription-app nivetha123/prescription-app

---

🔴 **STEP 9: Push to DockerHub**

1. Create account in DockerHub
2. Run:

```
docker login
docker push yourname/prescription-app
```

🔴 **STEP 10: Create deployment.yaml**

👉 Create file:

deployment.yaml

👉 Paste:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prescription-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: prescription
  template:
    metadata:
      labels:
        app: prescription
    spec:
      containers:
      - name: app
        image: yourname/prescription-app:latest
        ports:
        - containerPort: 8080
```

🔴 **6. Kubernetes Service**

📄 service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: prescription-service
spec:
  type: NodePort
  selector:
    app: prescription
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
      nodePort: 30007
```

🔴 **STEP 11: Run Kubernetes**

kubectl apply -f deployment.yaml

kubectl apply -f service.yaml

if deployment.yaml already exist

✅ **SIMPLE FIX (BEST METHOD)**

🔴 **STEP 1: Delete old deployment**

Run:

kubectl delete deployment prescription-app

🔴 **STEP 2: Apply again**

kubectl apply -f deployment.yaml

🔍 **STEP 3: Check status**

kubectl get pods

kubectl get svc

kubectl logs prescription-app-b444f5495-85mns

✅ **EXPECTED OUTPUT**

You should see something like:

prescription-app-xxxxx Running

🔴 **STEP 1: Create GitHub Account**

👉 Go to:
🌐 https://github.com

👉 Sign up (if not already)

🔴 **STEP 2: Create Repository**

👉 Click:

- **New Repository**

Fill:

Field	Value
Repository name	prescription-app
Visibility	Public
Initialize	❌ Don't check anything

👉 Click **Create Repository**

🔴 **STEP 3: Open Terminal in Your Project**

You are already here:

C:\Users\Nivetha Srinivasan\OneDrive\Documents\Agile\prescription-app

Good 👍

🔴 **STEP 4: Initialize Git**

Run:

```
git init
```

🔴 **STEP 5: Add Files**

```
git add .
```

👉 This adds all files

🔴 **STEP 6: Commit Files**

```
git commit -m "first commit"
```

🔴 **STEP 7: Connect to GitHub**

👉 Copy your repo URL from GitHub

Example:

https://github.com/nivetha123/prescription-app.git

Run:

```
git remote add origin https://github.com/YOUR_USERNAME/prescription-app.git
```

🔴 **STEP 8: Push Code**

```
git branch -M main
git push -u origin main
```

🔍 **VERIFY**

👉 Open GitHub
👉 You should see all files:

✓ src/
✓ pom.xml
✓ Dockerfile
✓ deployment.yaml

🔴 **STEP 12: Jenkins Setup (Simple)**

Open Terminal : java -jar jenkins.war

🔴 **PART 1: Open Jenkins**

👉 Open browser and go:

http://localhost:8080

🔴 **PART 2: Create New Pipeline Job**

👉 Click:

1. **New Item**
2. Enter name:

prescription-pipeline

3. Select:
   👉 **Pipeline**
4. Click **OK**

🔴 **PART 3: Configure Pipeline**

Scroll down to:

👉 **Pipeline Section**

Change:

Definition → Pipeline script

🔴 **PART 4: Paste Pipeline Code**

👉 Delete everything → Paste this:

```
pipeline {
  agent any

  environment {
    DOCKER_IMAGE = "YOUR_DOCKER_USERNAME/prescription-app:latest"
  }

  stages {

    stage('Clone Code') {
      steps {
        git branch: 'main',
        url: 'https://github.com/YOUR_GITHUB_USERNAME/YOUR_REPO.git'
      }
    }

    stage('Build & Test') {
      steps {
        bat 'mvn clean test'
      }
    }

    stage('Package') {
      steps {
        bat 'mvn package -DskipTests'
      }
    }

    stage('Build Docker Image') {
      steps {
        bat 'docker build -t %DOCKER_IMAGE% .'
      }
    }

    stage('Push Docker Image') {
      steps {
        withCredentials([usernamePassword(
          credentialsId: 'dockerhub-creds',
          usernameVariable: 'DOCKER_USER',
          passwordVariable: 'DOCKER_PASS'
        )]) {
```

```
            bat '''
            docker login -u %DOCKER_USER% -p %DOCKER_PASS%
            docker push %DOCKER_IMAGE%
            '''
          }
        }
      }

      stage('Deploy to Kubernetes') {
        steps {
          bat 'kubectl apply -f deployment.yaml'
        }
      }
    }
}
```

🔴 **PART 5: IMPORTANT CHANGES YOU MUST DO**

⚠️ Replace these 2 things:

✅ **1. Docker Username**

Change:

YOUR_DOCKER_USERNAME

👉 Example:

nivetha123/prescription-app:latest

✅ **2. GitHub Repository Link**

Change:

https://github.com/YOUR_GITHUB_USERNAME/YOUR_REPO.git

👉 Example:

https://github.com/nivetha123/prescription-app.git

🔴 **PART 6: Add DockerHub Credentials (VERY IMPORTANT)**

👉 Go to:

Manage Jenkins → Credentials

Steps:

1. Click:
   👉 **(global) → Add Credentials**
2. Fill:

Field	Value
Kind	Username with password
Username	Your DockerHub username
Password	Your DockerHub password
ID	dockerhub-creds
Description	DockerHub login

👉 Click **Create**

🔴 **PART 7: Save Pipeline**

👉 Click:

Save

🔴 **PART 8: Run Pipeline**

👉 Click:

Build Now

Output :

🔴 **1. Maven Test Output (VERY IMPORTANT)**

👉 Command you ran:

mvn clean test

👉 Paste this part:

```
[INFO] Running PrescriptionTest
[INFO] Tests run: 2, Failures: 0, Errors: 0
[INFO] BUILD SUCCESS
```

✓ This proves:

- Your code works
- JUnit testing is successful

🔴 **2. Maven Package Output**

👉 Command:

mvn package

👉 Paste:

[INFO] Building jar: target/prescription-app-1.0.jar
[INFO] BUILD SUCCESS

✓ This proves:

- Your application is built correctly

🔴 **3. Docker Build Output**

👉 Command:

docker build -t nivetha140805/prescription-app .

👉 Paste:

=> naming to docker.io/nivetha140805/prescription-app:latest
=> DONE

✓ This proves:

- Docker image created

🔴 **4. Docker Push Output**

👉 Command:

docker push nivetha140805/prescription-app

👉 Paste:

latest: digest: sha256:xxxxx size: xxxx

✓ This proves:

- Image uploaded to DockerHub

🔴 **5. Kubernetes Deployment Output**

👉 Command:

kubectl apply -f deployment.yaml

👉 Paste:

deployment.apps/prescription-app created
service/prescription-service created

✓ This proves:

- App deployed in Kubernetes

🔴 **6. Kubernetes Running Status (VERY IMPORTANT)**

👉 Command:

kubectl get pods

👉 Paste:

prescription-app-xxxxx Running

👉 Command:

kubectl get svc

👉 Paste:

prescription-service NodePort

✓ This proves:

- App is running successfully

🔴 **7. Jenkins Pipeline Output (MOST IMPORTANT)**

👉 Paste final part:

Finished: SUCCESS

OR screenshot of:

✓ All stages green:

- Clone ✓
- Test ✓
- Build ✓
- Docker ✓
- Deploy ✓

✅ **1. Java Program Execution**

✓ Covered

👉 You created:

- PrescriptionService.java

👉 What it does:

- Validates medicine name
- Checks dosage
- Combines logic in one method

👉 Proof (what you show):

- Your Java code
- Maven compile/build success

✓ So this step is **100% completed**

✅ 2. JUnit Testing

✓ Covered

👉 You created:

- PrescriptionTest.java

👉 What it does:

- Tests valid medicine ✓
- Tests invalid medicine ✓
- Tests dosage ✓

👉 Output you got:

Tests run: 2, Failures: 0

✓ So:

- Testing is automated
- Code is verified

✓ This step is **fully completed**

✅ 3. CI/CD Pipeline (Jenkins)

✓ Covered

👉 Your Jenkins pipeline stages:

Clone → Build → Test → Package → Docker → Deploy

👉 What happens:

- Code pulled from GitHub
- Tests run automatically
- Build happens
- Docker image created
- Deployment triggered

👉 Final output:

Finished: SUCCESS

✓ This is exactly what CI/CD means:

Continuous Integration + Continuous Deployment

✓ This step is **perfectly done**

✅ **4. Docker & Kubernetes Deployment**

✓ **Covered**

Docker:

- Created image:

nivetha140805/prescription-app

✓ Containerization done

Kubernetes:

- Used deployment.yaml
- Deployed app

👉 Commands:

```
kubectl apply -f deployment.yaml
kubectl get pods
```

✓ App is running in cluster